

HVE-Coder Architecture Proposal

Dual-backend Hermes layout on NVIDIA DGX Spark
Ollama + Qwen 3.8 · vLLM + Nemotron 3.5 Lightning

DECISION

Move the HVE-coder profile onto Lightning served by vLLM.

KEEP

Daily Hermes profile on Ollama with Qwen 3.8 27B.

CONSTRAINT

One GB10, 128 GB unified memory — cap vLLM or they collide.

GAIN

Faster coder tool loops without giving up Qwen as the general brain.

1. Purpose and decision

This note records the proposed inference layout for two Hermes profiles on the NVIDIA DGX Spark. The daily HVE agent stays on Ollama with Qwen 3.8 27B. The HVE-coder profile, created for coding, moves onto NVIDIA Nemotron 3.5 Lightning served by vLLM with DSpark speculative decoding.

The two profiles do not share memory, skills, or sessions. Their HTTP ports do not collide. What they do share is the GB10: one GPU and one 128 GB unified memory pool. The architecture therefore treats engine isolation as a configuration problem and memory as the real constraint.

Decision. Adopt the split. Do not run Lightning inside Ollama as a second copy. Cap vLLM so Qwen can stay resident. Treat simultaneous generation as “both stay responsive,” not “twice the tokens per second.”

2. Why this split

Nemotron 3.5 Lightning is a 30B-total / ~3B-active hybrid Mamba–MoE model distilled for the execution layer of always-on agents: tool calls, patches, retries, test loops. NVIDIA ships it with a Spark-specific draft model (DSpark) and a vLLM tool parser aligned to coder-style function calls. Qwen 3.8 27B remains the stronger general and multimodal local brain (vision, harder planning, higher intelligence-index scores). Putting both jobs on one model wastes that difference.

Job	Profile	Engine	Model	Why
Planning, docs, vision, general HVE	Daily Hermes	Ollama :11434	Qwen 3.8 27B Q4	Higher general intelligence; vision; already wired
Code, tests, patches, tool storms	HVE-coder	vLLM :8000	Lightning NVFP4 + DSpark	Built for harness loops; batches concurrent tool calls
Shell / git / pytest / sandbox	Either	Host OS	—	CPU and disk. Does not need a second LLM engine

Figure is implicit: one job, one engine, one model. No double-loading of Lightning.

3. System layout

Hermes profiles are separate home directories. HVE-coder gets its own config, SOUL.md, skills, and working directory so two agents editing at once cannot stomp files. Each profile points at a different OpenAI-compatible base URL.

NVIDIA DGX SPARK · GB10 Grace Blackwell · 128 GB unified LPDDR5X · one GPU

Single physical pool. CPU, GPU, Ollama, vLLM, Hermes, and sandboxes allocate from the same 128 GB.

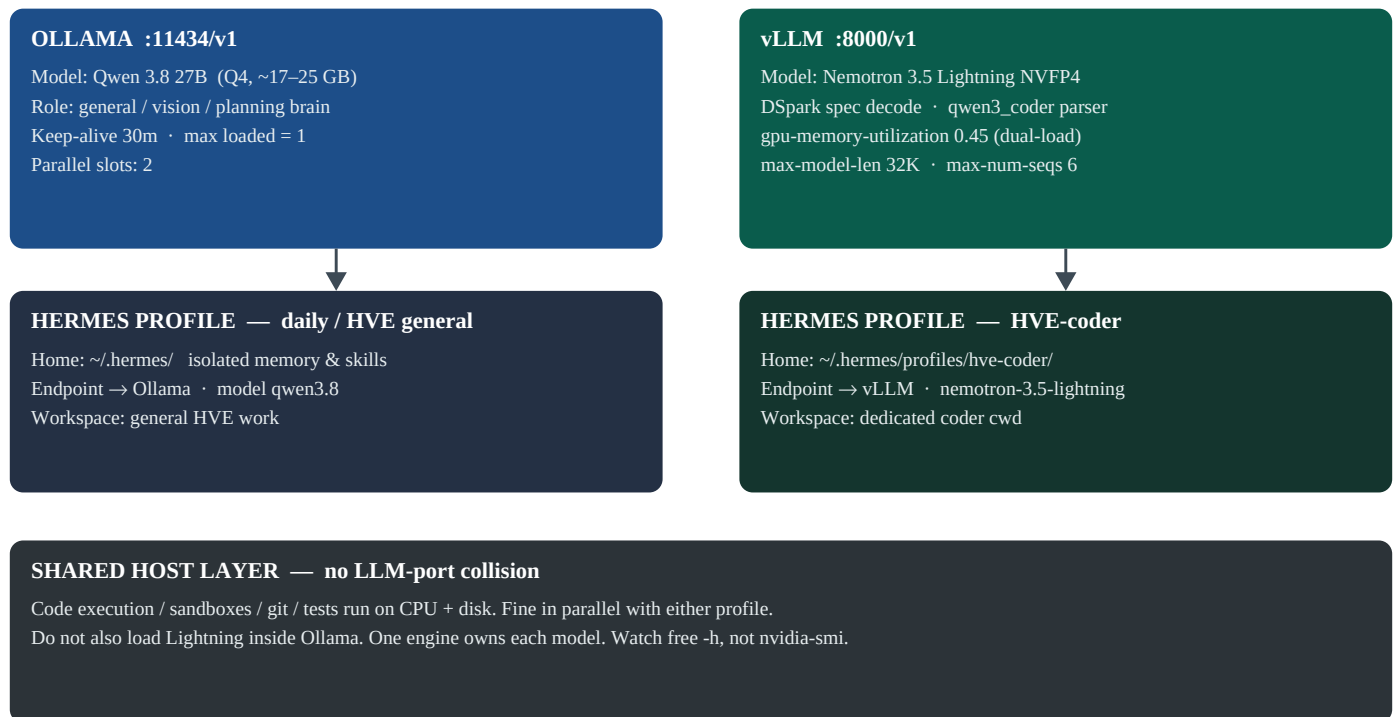


Figure 1. Dual-backend layout on a single DGX Spark.

Layer	Isolated?	Notes
Hermes profile state	Yes	Separate ~/.hermes vs ~/.hermes/profiles/hve-coder — memory, skills, gateway, SOUL.md
HTTP endpoints	Yes	Ollama 127.0.0.1:11434/v1 and vLLM 127.0.0.1:8000/v1
Model weights	Yes, if configured	Qwen lives only in Ollama. Lightning lives only in vLLM. Do not pull Lightning into Ollama.
Code execution	Mostly	Isolate cwd per profile. CPU contention is mild next to GPU contention.
GB10 unified memory + SMs	No	One pool. vLLM default 0.9 utilization will starve Ollama. Cap it.

Table 1. What is isolated versus what is shared.

4. Memory architecture on GB10

Unified memory is not “128 GB of VRAM plus system RAM.” It is one LPDDR5X pool. vLLM pre-allocates KV cache from whatever it thinks is free at startup. On Spark that default often reserves 80–100 GB. When Ollama later grows a Qwen context, pages get reclaimed under vLLM’s feet. The failure mode is latency collapse, not a clean out-of-memory error. Watch free -h, not nvidia-smi.

Slice	Dual-load target	Coder-only (Qwen unloaded)
OS + two Hermes gateways + sandboxes	~15–20 GB	~12–16 GB
Ollama Qwen 3.8 Q4 + 32–64K context	~22–30 GB	0 (stopped)
vLLM Lightning NVFP4 + DSpark + KV	~40–50 GB hard cap	~75–95 GB
gpu-memory-utilization	0.40–0.50	0.75–0.80
max-model-len	32K (64K if slack)	64K–128K as needed
max-num-seqs	4–8	8–16
Headroom to keep	≥20 GB	≥15 GB

Table 2. Unified-memory budget. Dual-load is the default operating mode.

Operating rule. Start dual-load at 0.45 utilization. If free -h stays stable and Ollama does not page while HVE-coder is busy, raise to 0.55. Never start at 0.85 while Qwen is resident. When HVE-coder owns the box for a long session, stop Qwen in Ollama and raise vLLM.

5. Throughput reality

Two engines on one GPU do not produce two full-speed streams. Decode is time-sliced across the same SMs and the same memory controllers. The value of vLLM for HVE-coder is internal batching: several short tool calls from one or two coder sessions stay in one continuous batch with prefix caching and DSpark. That is the workload Lightning was built for.

Pattern	Expected result
HVE-coder alone on vLLM	Best tok/s. DSpark + prefix cache. Raise utilization.
Daily profile chatting, coder idle	Ollama Qwen at full local speed. vLLM sits on reserved KV — that is the tax of dual-load.
Both generating at once	Both stay alive. Combined tok/s is less than the sum. Acceptable for mixed work; bad for a timed coding burst.
Two HVE-coder sessions	Keep them on the same vLLM. Do not stand up a second Lightning in Ollama.
Lightning in both Ollama and vLLM	Reject. Doubles weight footprint for no batching gain.

Table 3. Concurrency patterns.

6. Profile and endpoint wiring

Each Hermes profile uses a custom OpenAI-compatible provider. Leave API keys empty. Give HVE-coder a dedicated repository root so file tools cannot wander into the other profile's tree.

Daily / general profile — Ollama

```
model.default: qwen3.8:27b
provider: custom
base_url: http://127.0.0.1:11434/v1
```

HVE-coder profile — vLLM

```
model.default: nemotron-3.5-lightning
provider: custom
base_url: http://127.0.0.1:8000/v1
terminal.cwd: /home/.../hve-coder-workspace
```

Optional Hermes auxiliary slots on the daily profile can keep vision on Qwen and title/compression on a small Ollama tag so the general agent does not spend Qwen tokens on housekeeping. HVE-coder should keep auxiliary slots on Lightning unless a task explicitly needs vision.

7. Engine configuration

Ollama environment (host)

```
export OLLAMA_MAX_LOADED_MODELS=1
export OLLAMA_NUM_PARALLEL=2
export OLLAMA_KEEP_ALIVE=30m
```

One Qwen resident. Two parallel slots cover a chat turn plus a light tool call. Do not raise max loaded models to park Lightning beside Qwen.

vLLM — dual-load recipe (HVE-coder)

```
docker run -d --name vllm-hve-coder --gpus all --ipc=host --network=host \
-v $HOME/.cache/huggingface:/root/.cache/huggingface \
-e VLLM_NVFP4_GEMM_BACKEND=marlin \
vllm/vllm-openai:v0.27.1 \
--model nvidia/NVIDIA-Nemotron-3.5-Lightning-30B-A3B-NVFP4 \
--served-model-name nemotron-3.5-lightning \
--port 8000 --moe-backend marlin --kv-cache-dtype fp8 \
--mamba-backend flashinfer --enable-prefix-caching \
--gpu-memory-utilization 0.45 --max-model-len 32768 --max-num-seqs 6 \
--speculative_config.method dspark \
--speculative_config.model nvidia/NVIDIA-Nemotron-3.5-Lightning-30B-A3B-NVFP4-DSpark \
--speculative_config.num_speculative_tokens 3 \
--reasoning-parser nemotron_v3 --enable-auto-tool-choice \
--tool-call-parser qwen3_coder
```

The qwen3_coder tool parser is required. Lightning’s function-call training matches that parser in the current vLLM Nemotron recipe. DSpark is the Spark-recommended speculator; MTP/DFlash are fallbacks if acceptance rate is poor on a given prompt mix.

8. Operating modes

Mode	When	Actions
A — Dual resident	Normal HVE day: chat + occasional code	Both engines up. vLLM at 0.45. Qwen keep-alive 30m.
B — Coder burst	Focused implementation session	ollama stop on Qwen. Restart vLLM at 0.75–0.80, 64K ctx.
C — General only	No coding for hours	Leave vLLM running (warm) or stop the container to reclaim ~40 GB.
D — Overnight agent	Long HVE-coder loops / tests	Mode B. Pin cwd. Log vLLM metrics. Cap max-num-seqs so one runaway session cannot fill the batch.

Table 4. Four modes. Default is A. Burst coding is B.

9. Risks and controls

Risk	Signal	Control
vLLM eats the pool	free -h collapses; Ollama stalls; no clean OOM	gpu-memory-utilization ≤ 0.50 while Qwen is loaded
CUDA graph / compile bloat	vLLM RSS jumps 10+ GB after first request	Prefer NVFP4 Marlin recipe; avoid leaving default 0.9 util

Risk	Signal	Control
Tool-call parse misses	Hermes retries; empty function names	Keep --tool-call-parser qwen3_coder and auto tool choice
Workspace collision	Two profiles edit the same tree	Separate cwd; no shared writable project root
Thinking tokens on every tool step	Coder feels slow despite high tok/s	Disable reasoning on short tool turns; enable for design questions
Perplexity Portable Computer later	Second stack wants Lightning too	Do not co-resident that app with this vLLM at 0.45 until measured

Table 5. Failure modes observed on GB10 dual-serve in the field.

10. Rollout

#	Step	Done when
1	Pull NVFP4 + DSpark weights; start vLLM at 0.45 / 32K / 6 seqs.	curl :8000/v1/models returns nemotron-3.5-lightning
2	Point HVE-coder profile at :8000. Keep daily profile on Ollama.	One coding turn completes with tool calls
3	Run a 20-minute mixed load: Qwen chat + coder edit/test loop.	free -h stable; neither engine restarts
4	Measure coder tool-call success and wall time vs previous Qwen-on-Ollama profile.	Written baseline in the HVE-coder notes
5	Document Mode B (unload Qwen, raise util) as a one-command script.	Burst script used once without manual flags
6	Only then consider raising dual-load util to 0.55 or context to 64K.	A week of Mode A with no paging

Table 6. Sequence. Do not skip the mixed-load hour.

11. What this is not

This is not Perplexity Portable Computer. That product will add Lightning later as an in-app model pick. This design is the HVE Hermes stack: local, profile-isolated, no token meter, explicit memory cap. It is also not a two-Spark cluster plan. If a second Spark appears, put vLLM+Lightning on one box and Ollama+Qwen on the other and drop the dual-load cap. Until then, the cap is the architecture.

One-line summary. HVE-coder → Lightning on vLLM :8000. Daily Hermes → Qwen 3.8 on Ollama :11434. Separate profile homes and cwds. Cap vLLM at ~45% of the unified pool whenever Qwen is resident. Batch coder work inside vLLM; do not dual-load the same model.

Document version 1.0 · 2 September 2026 · Hardware assumed: 1× DGX Spark GB10, 128 GB unified memory. Commands reflect public Nemotron 3.5 Lightning NVFP4 + DSpark recipes current as of late August 2026.